

SAC on Hopper

Reinforcement Learning Project

Muhammad Talal Faiz
Abdul Rafay Khan
Abdullah Aleem
SEECs, NUST

Hopper-V5

State Space

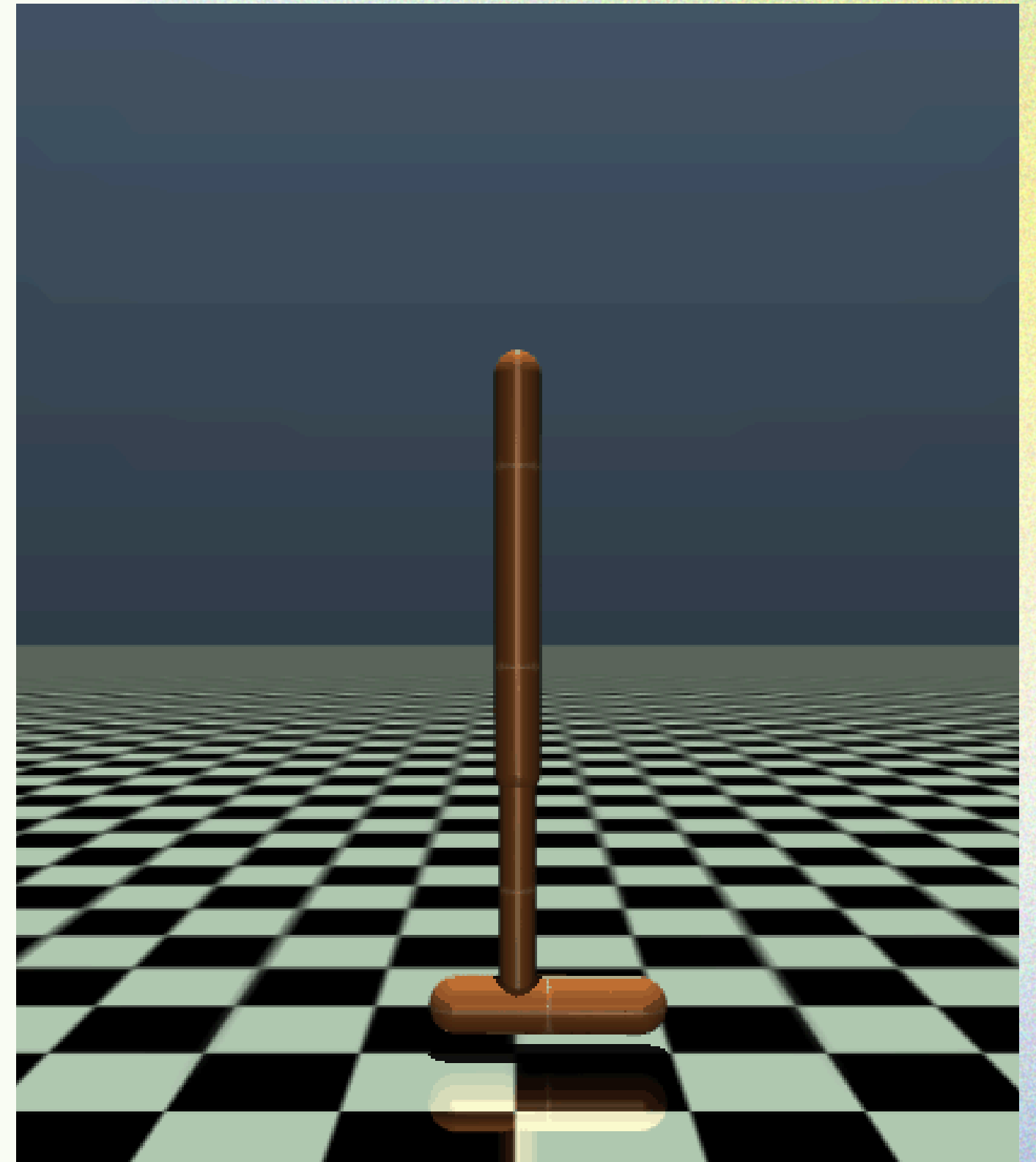
11-dimensional continuous state space

Observation Space `Box(-inf, inf, (11,), float64)`

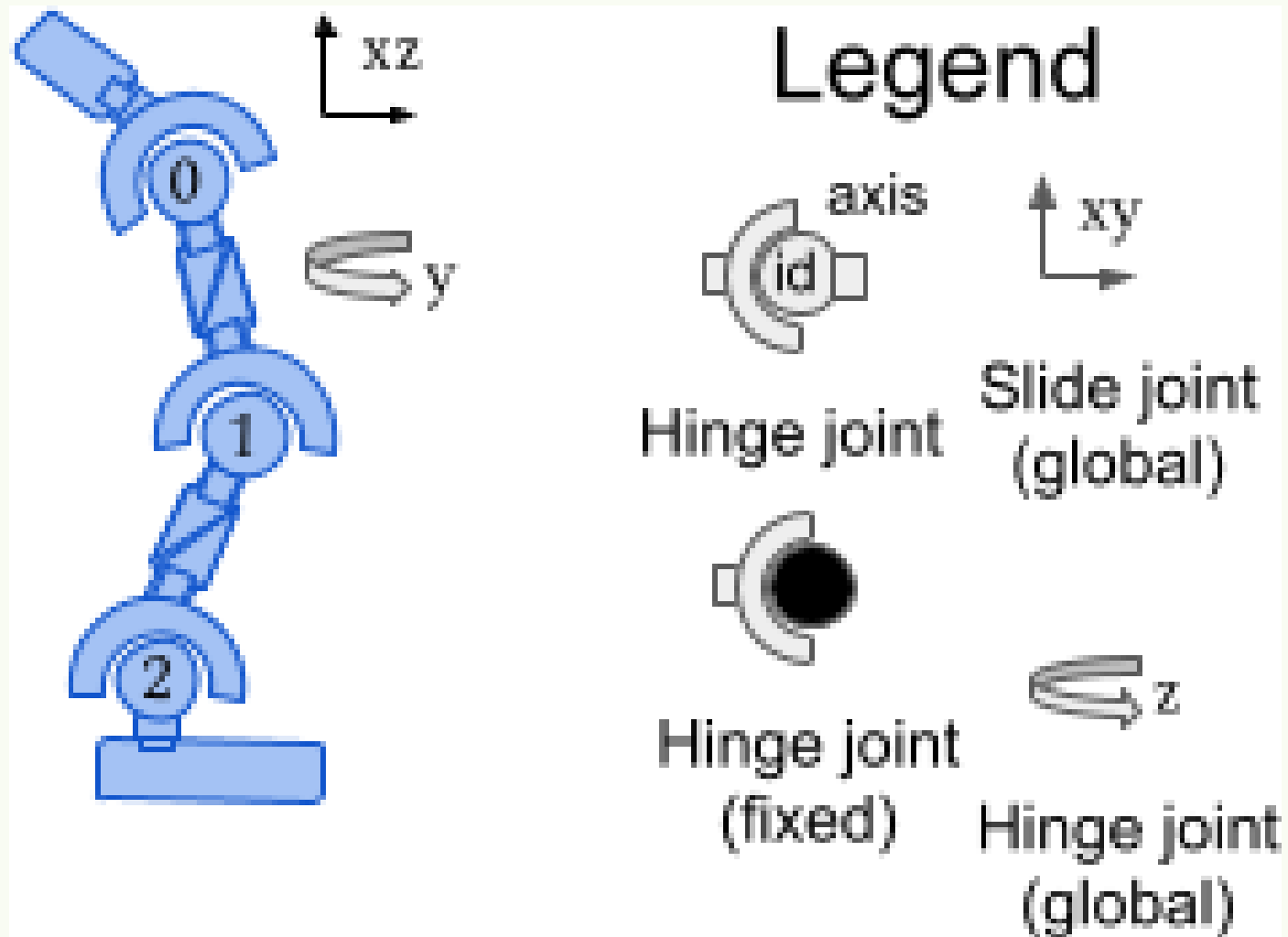
Action Space

3-dimensional continuous action space

Action Space `Box(-1.0, 1.0, (3,), float32)`



State & Action Spaces



3 Joint Torque-based Action

The hopper has 3 joints each having a torque value applied to it ranging from $[-1,1]$ which forms a 3-dimensional action

5	velocity of the x-coordinate of the torso
6	velocity of the z-coordinate (height) of the torso
7	angular velocity of the

11-dimensional State Space

The state space consist of two groups of state variables. Position & Velocity of the body parts of the Hopper.

Challenges

Continous Action Space

Continous State Space

Non-Linear Control Dynamics

Joint Coupling

Balancing

Moving Forward

Why SAC

[Back to Agenda Page](#)

Continuous Action Space

- Stochastic Policy that is Gaussian with mean & variance given by Neural Network

High Dimensionality

- Neural Networks for both Actor and Critic make it easy to generalise for High Dimensional States in Hopper

Sample Efficiency

- Off Policy method that makes use of Replay Buffers

Non-Linear Function Approximation

- Hopper is sensitive to action controls and has a Non-Linear relation with control dynamics hence rewards

SAC Innovation

[Back to Agenda Page](#)

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_{\pi}} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))]$$

Entropy Maximization

- Objective for policy has to maximize reward & entropy of policy (randomness of policy)

Actor-Critic Separate Policy & Value Network

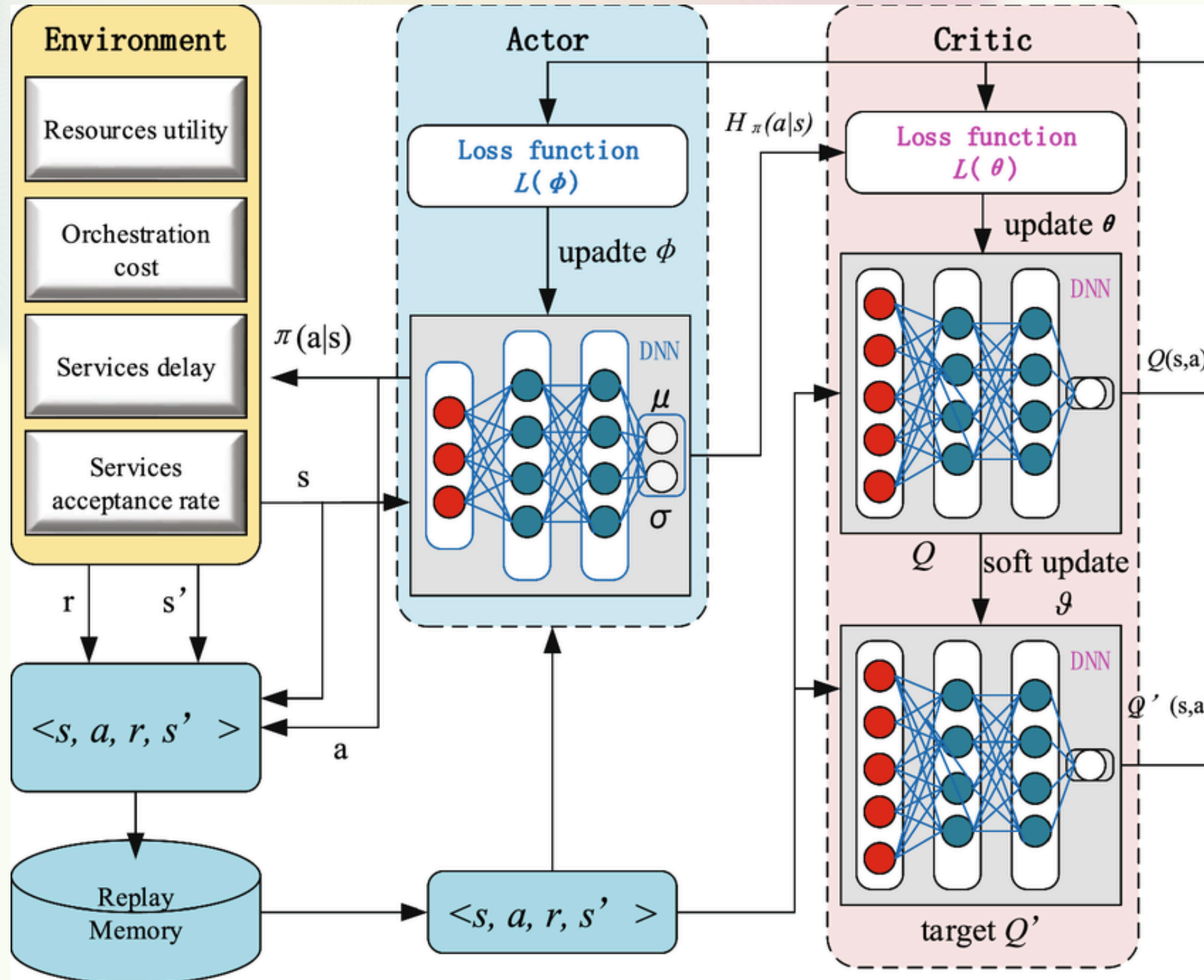
- Actor has to maximize reward & entropy
- Critic focuses on Q-Values

Off-Policy

- Reuse previous experience stored in buffers leading to sample efficiency

SAC Structure

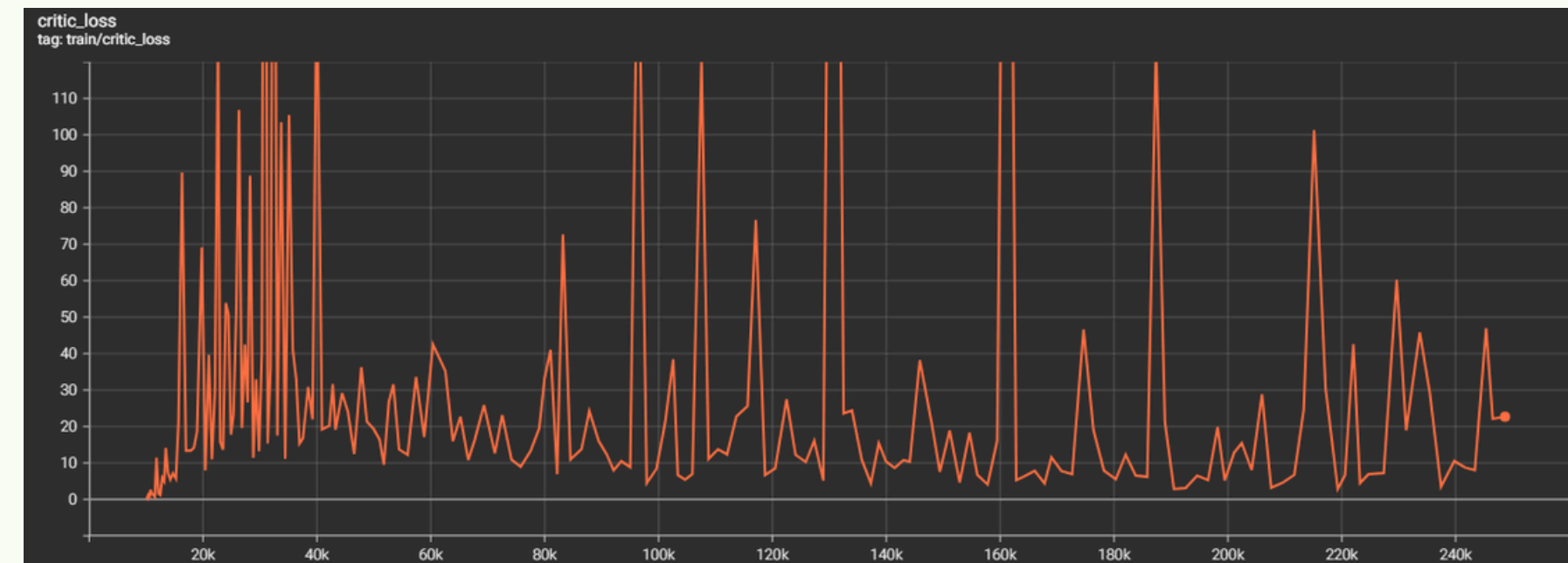
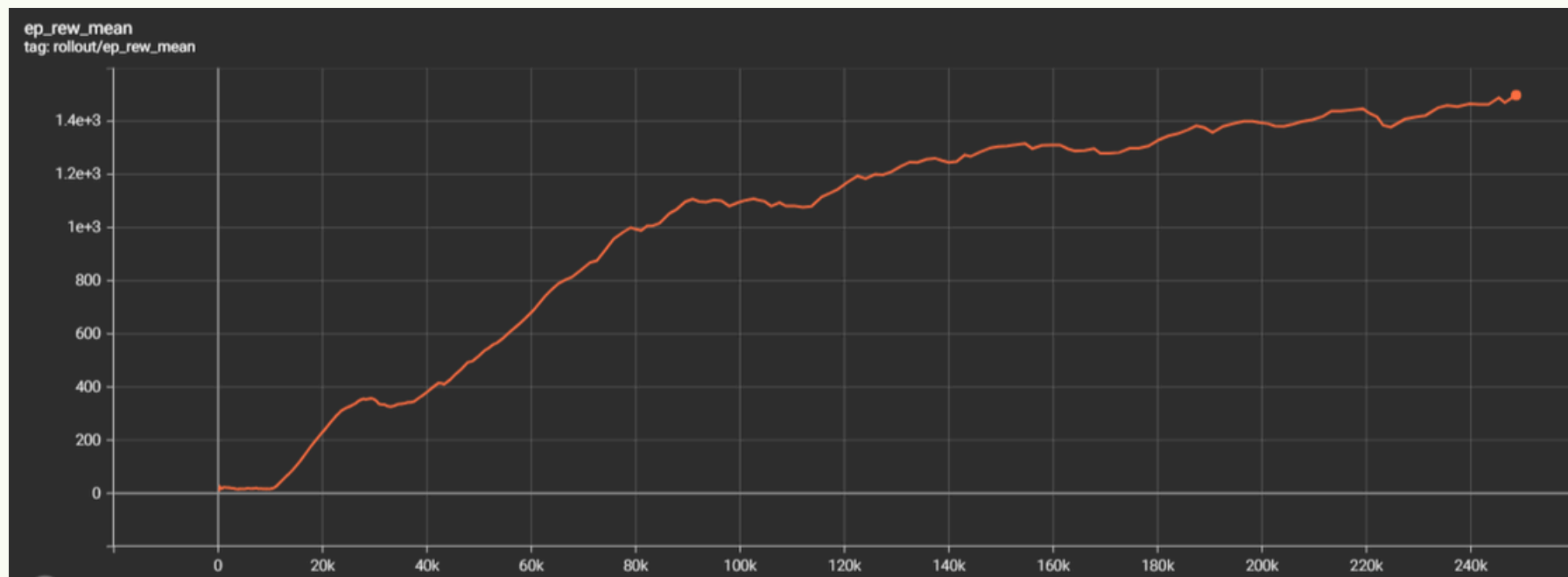
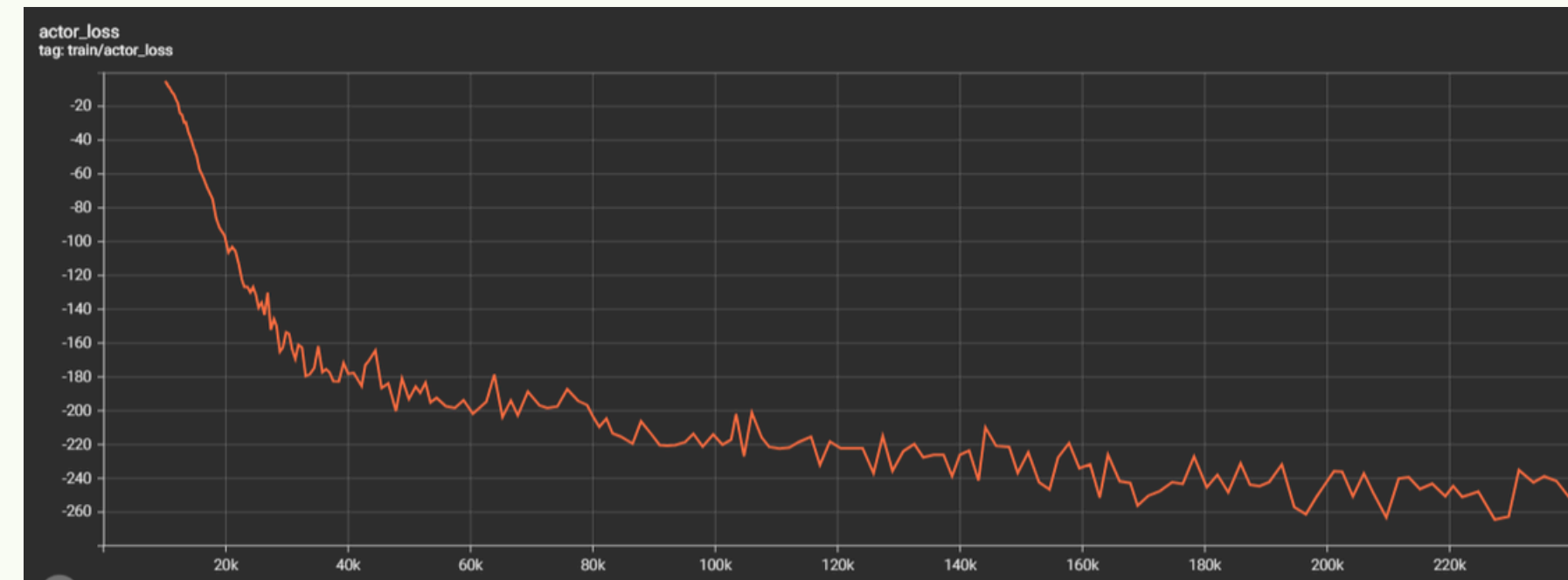
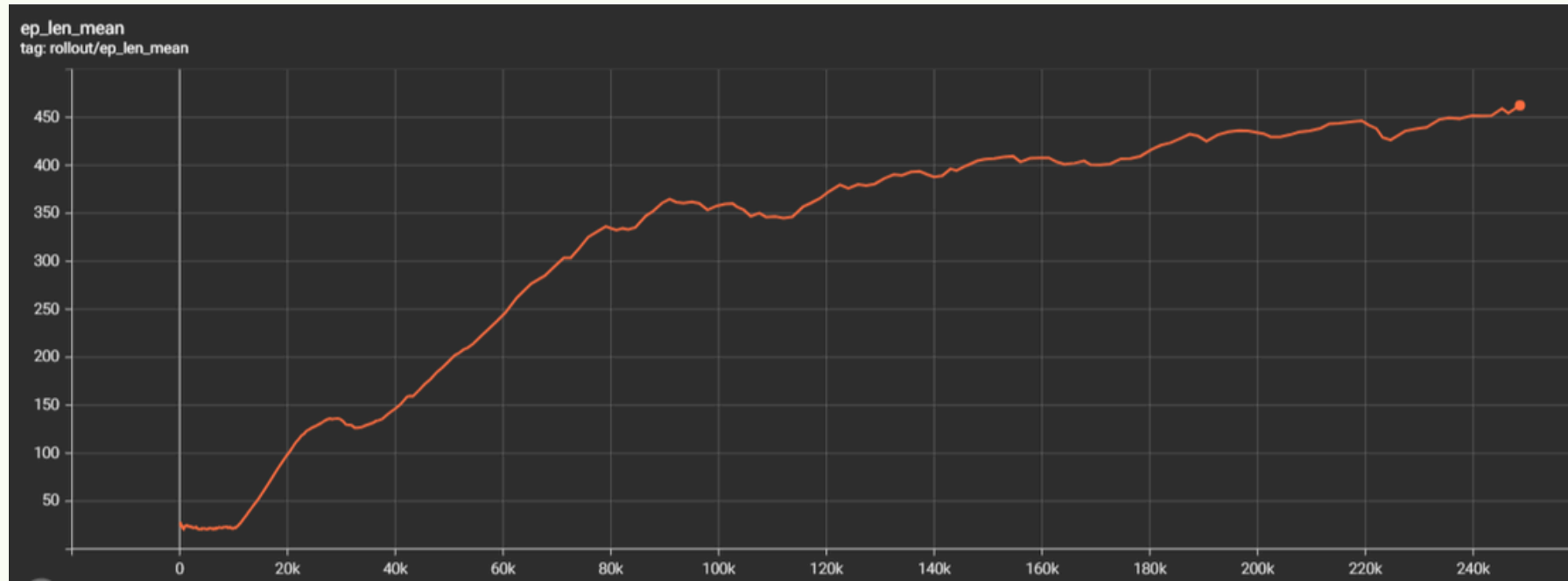
[Back to Agenda Page](#)



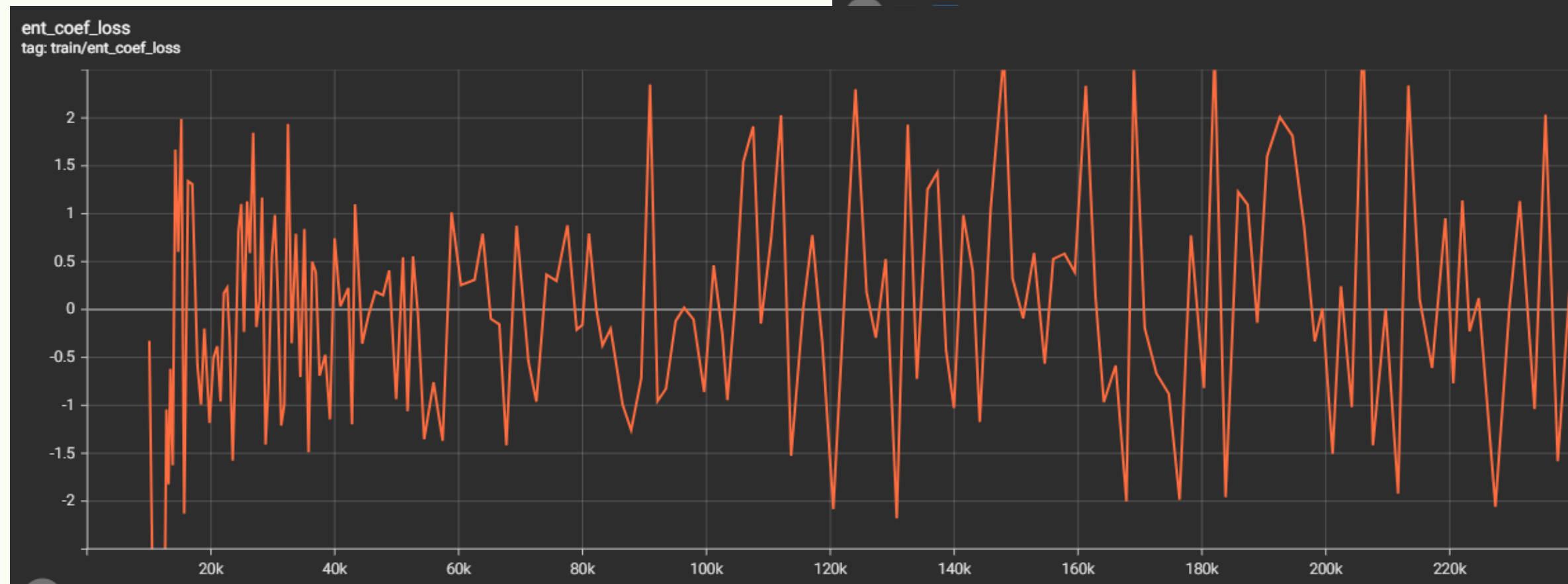
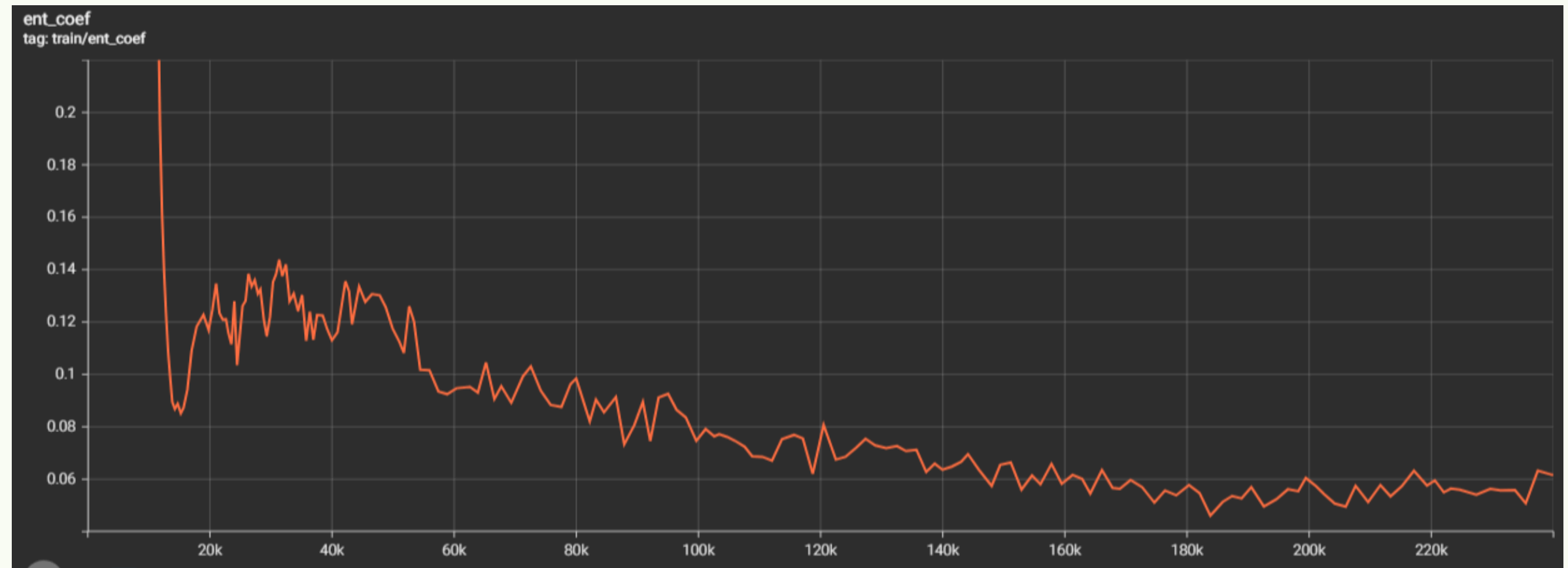
Algorithm 1 Soft Actor-Critic

Initialize parameter vectors $\psi, \bar{\psi}, \theta, \phi$.
for each iteration **do**
 for each environment step **do**
 $\mathbf{a}_t \sim \pi_{\phi}(\mathbf{a}_t | \mathbf{s}_t)$
 $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$
 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
 end for
 for each gradient step **do**
 $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_{\psi} J_V(\psi)$
 $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
 $\phi \leftarrow \phi - \lambda_{\pi} \hat{\nabla}_{\phi} J_{\pi}(\phi)$
 $\bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi}$
 end for
end for

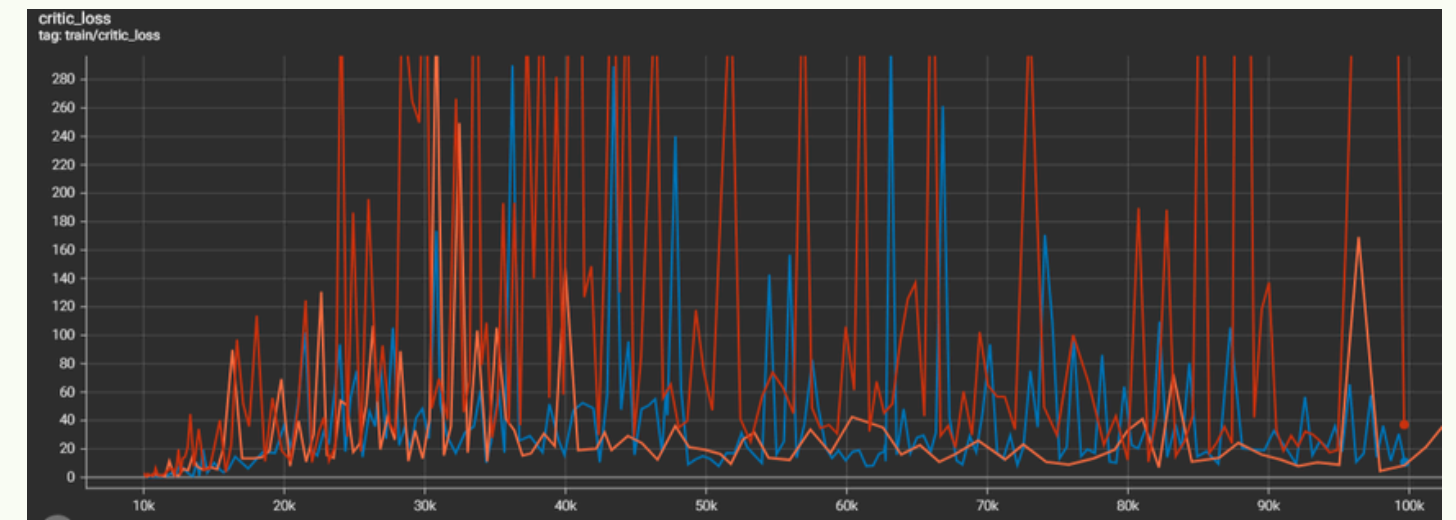
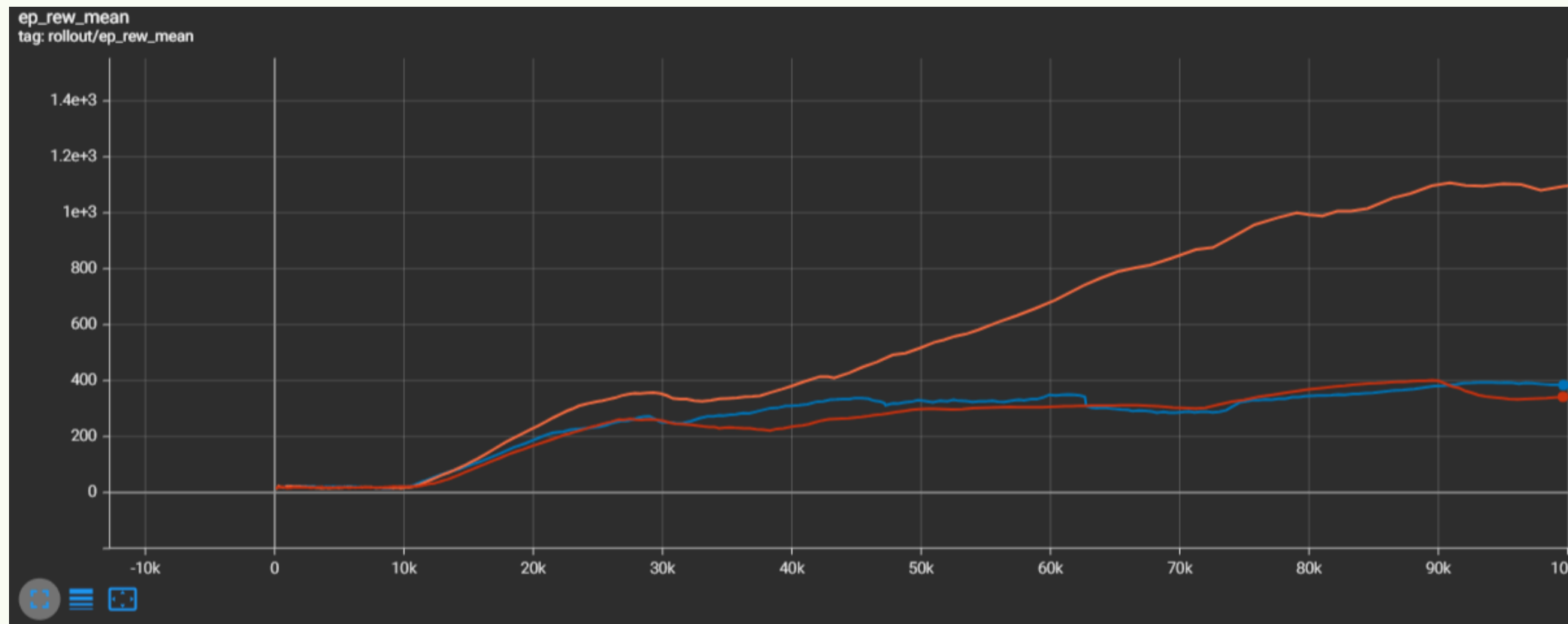
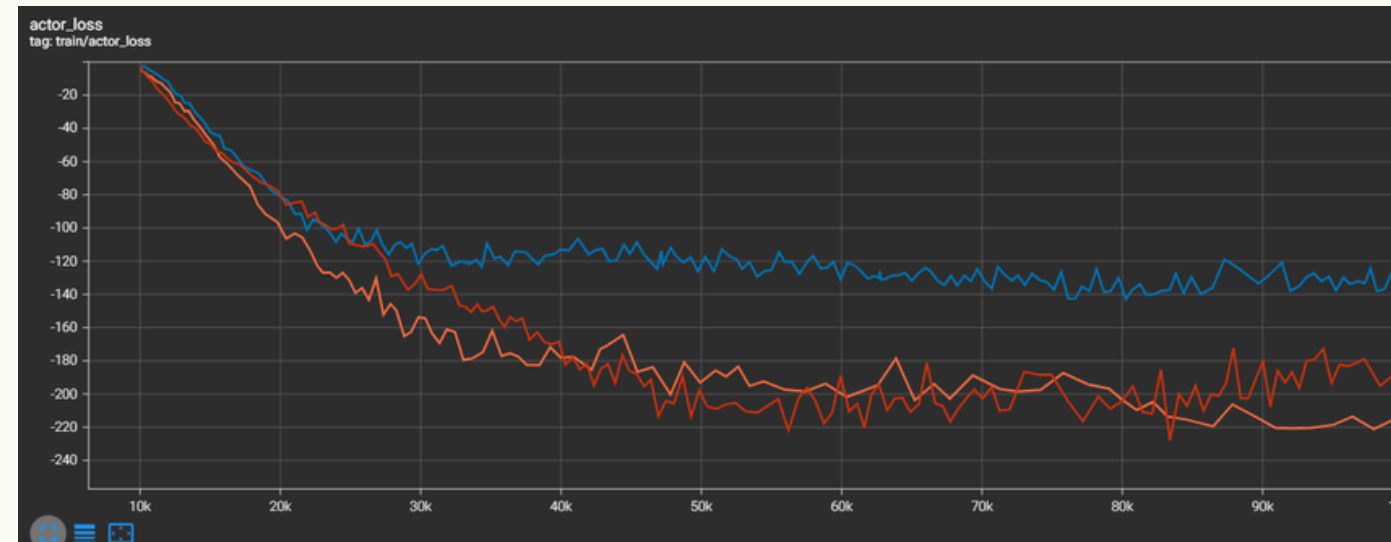
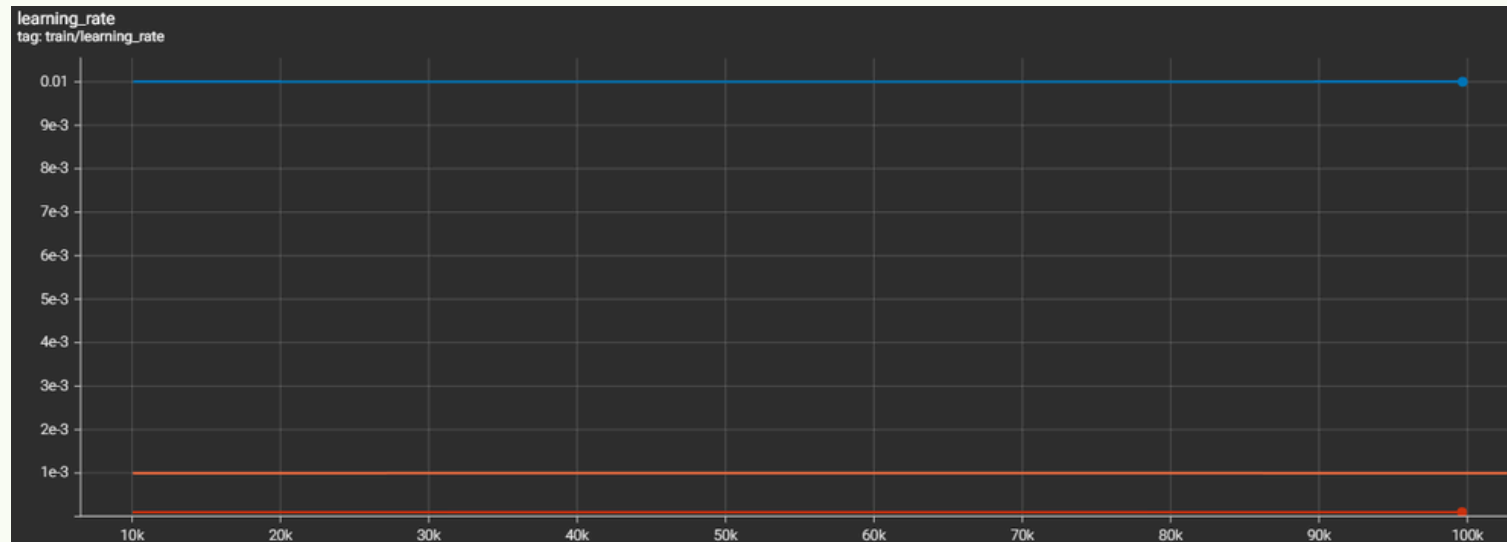
Default Results



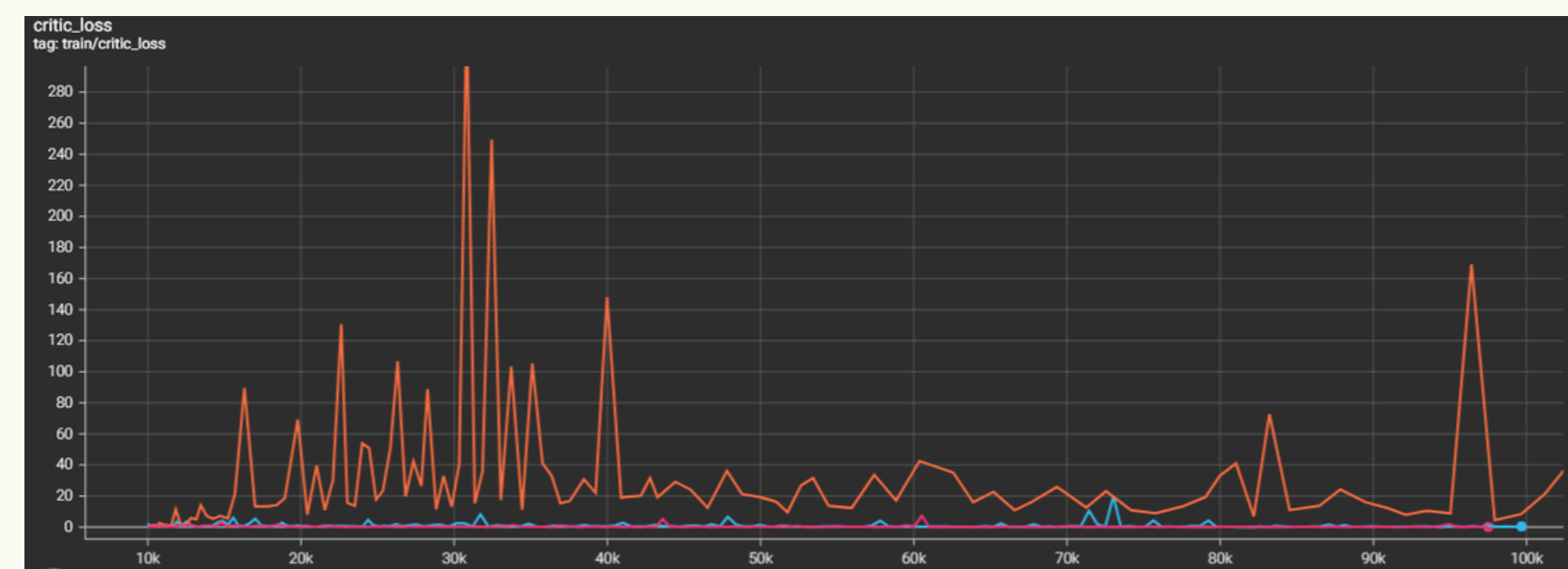
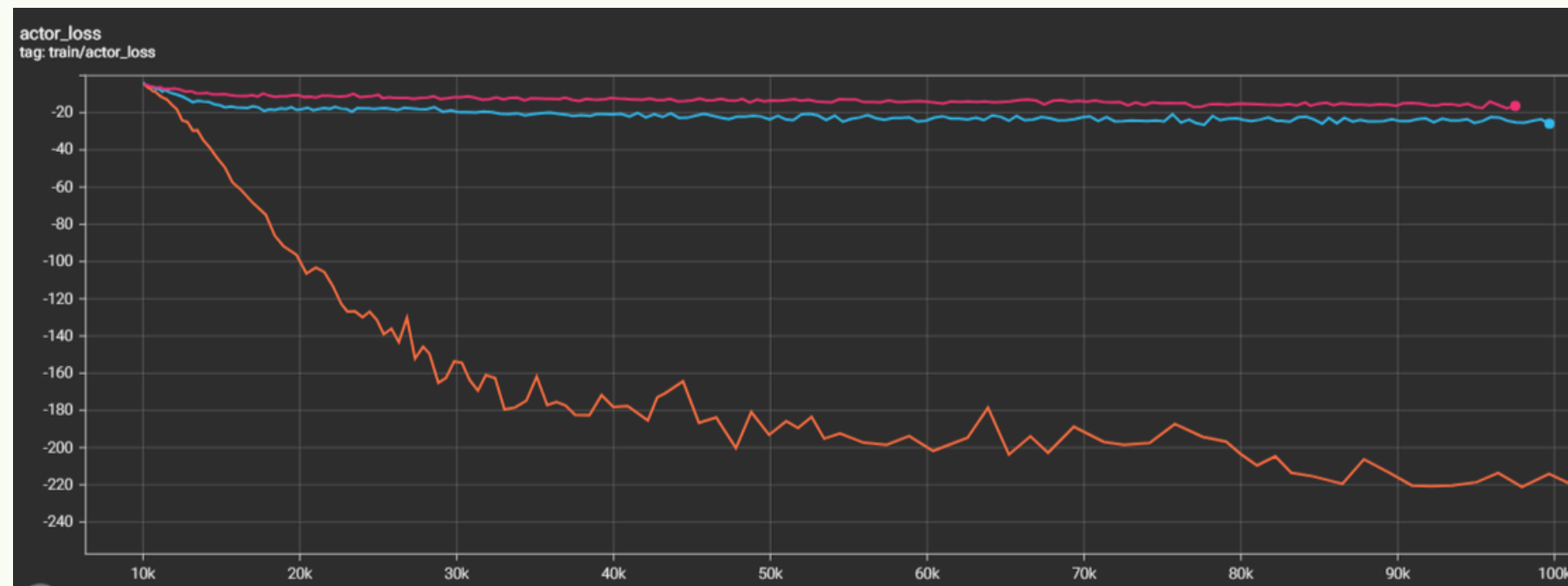
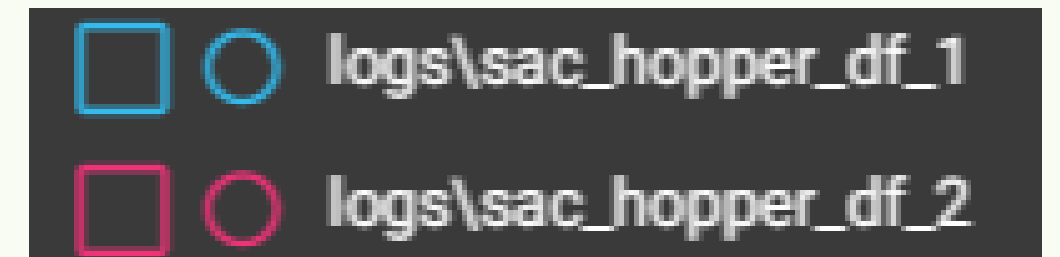
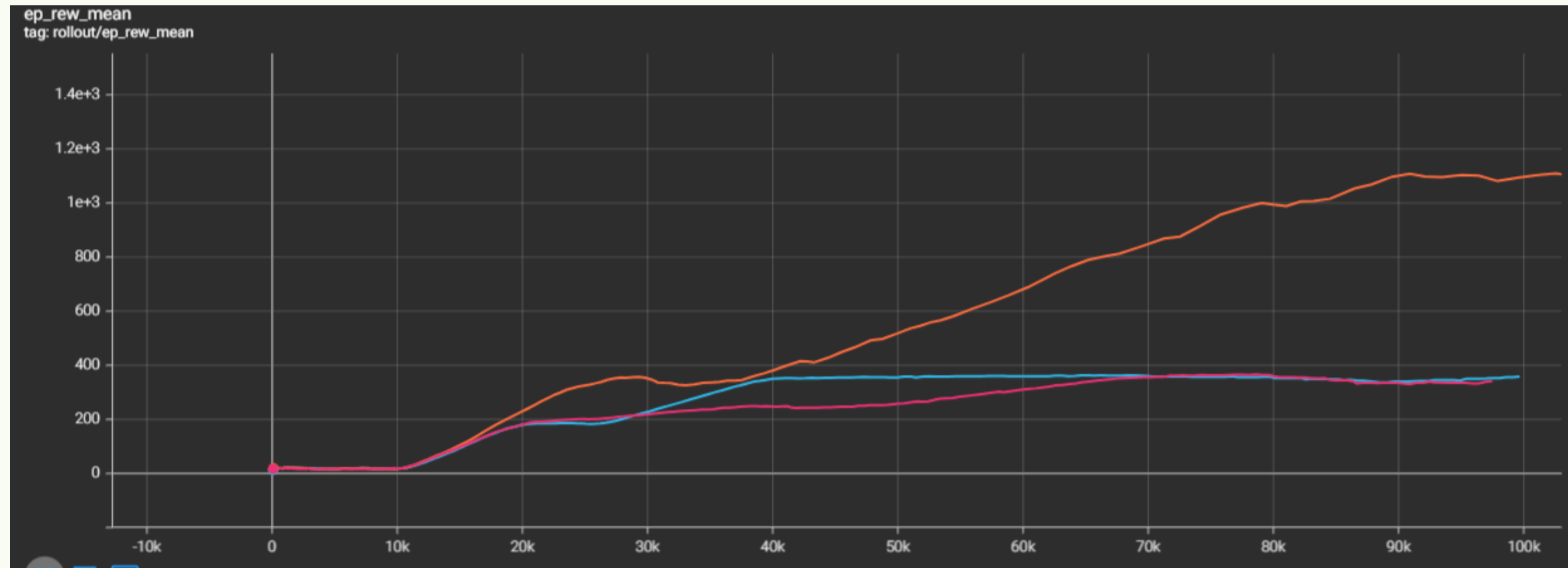
Default Results



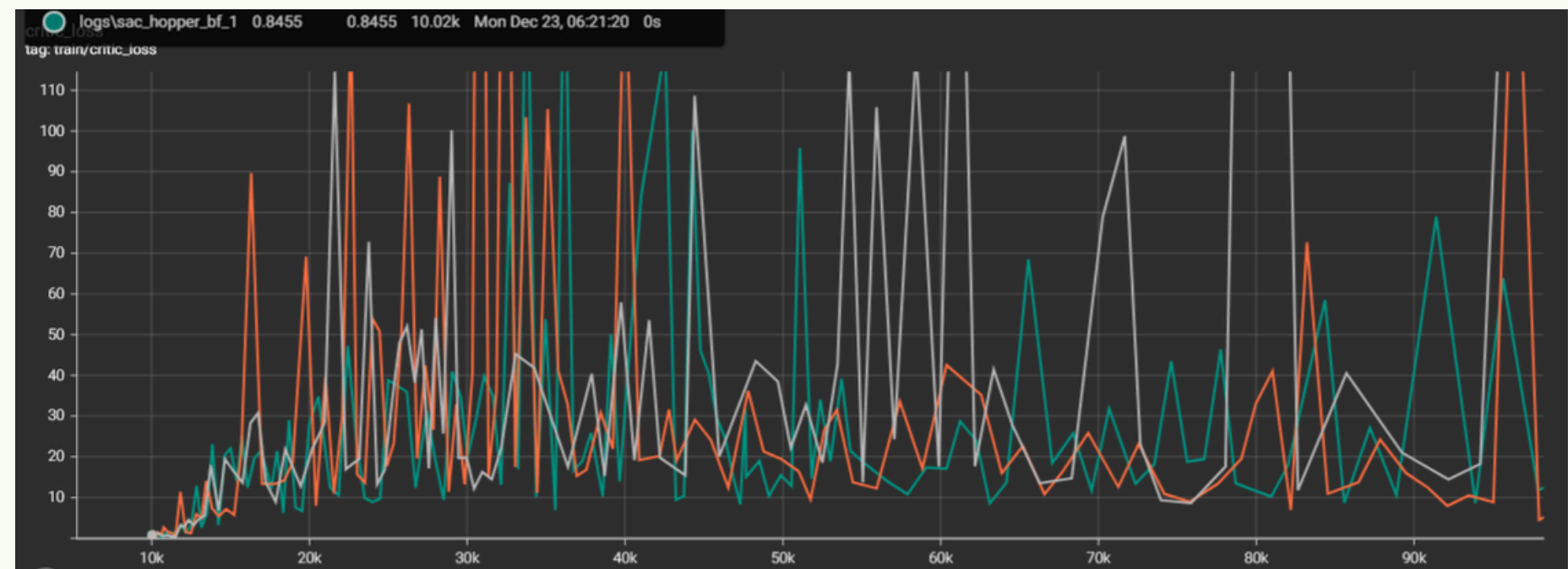
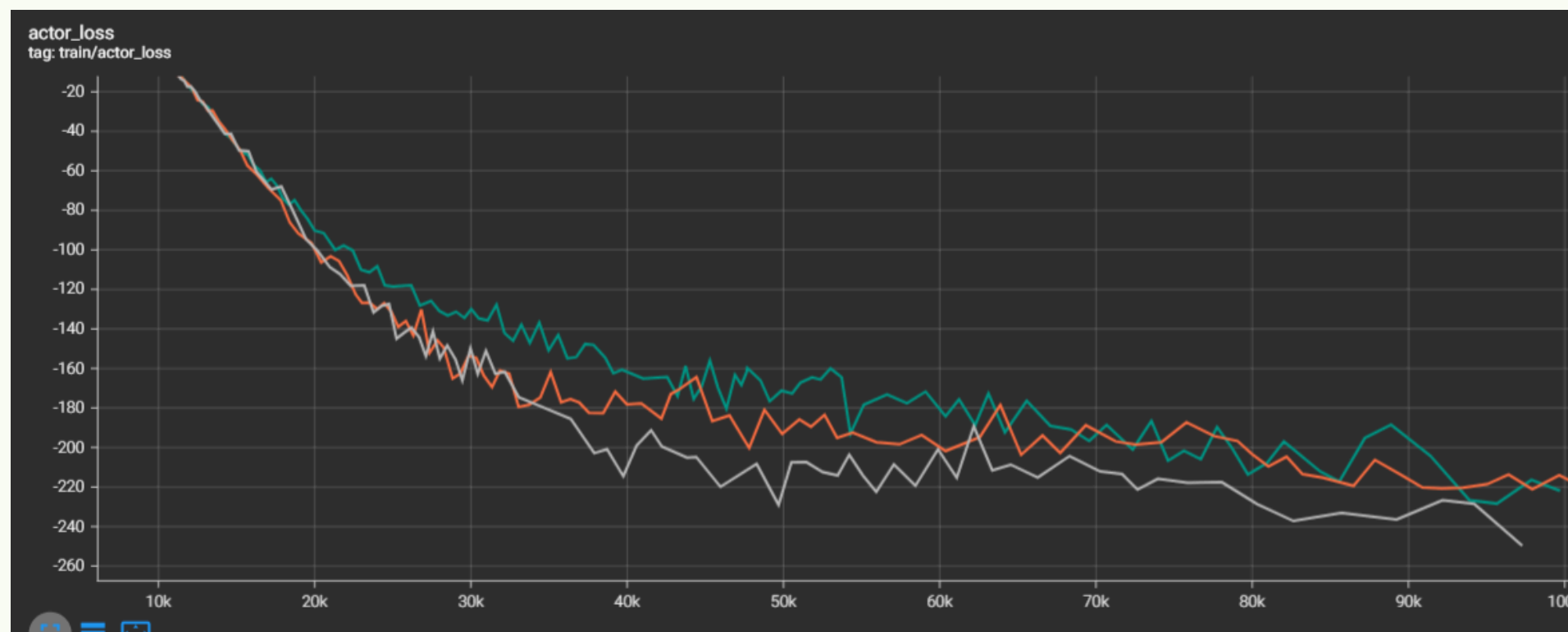
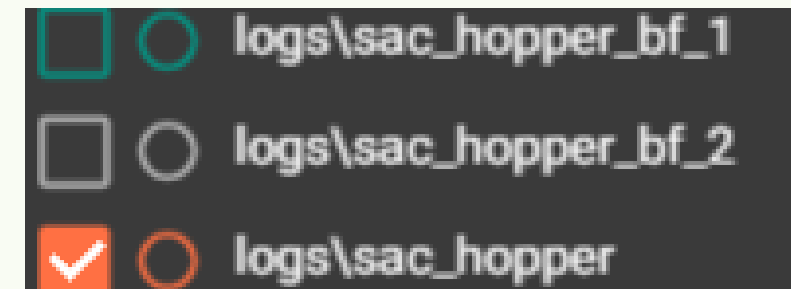
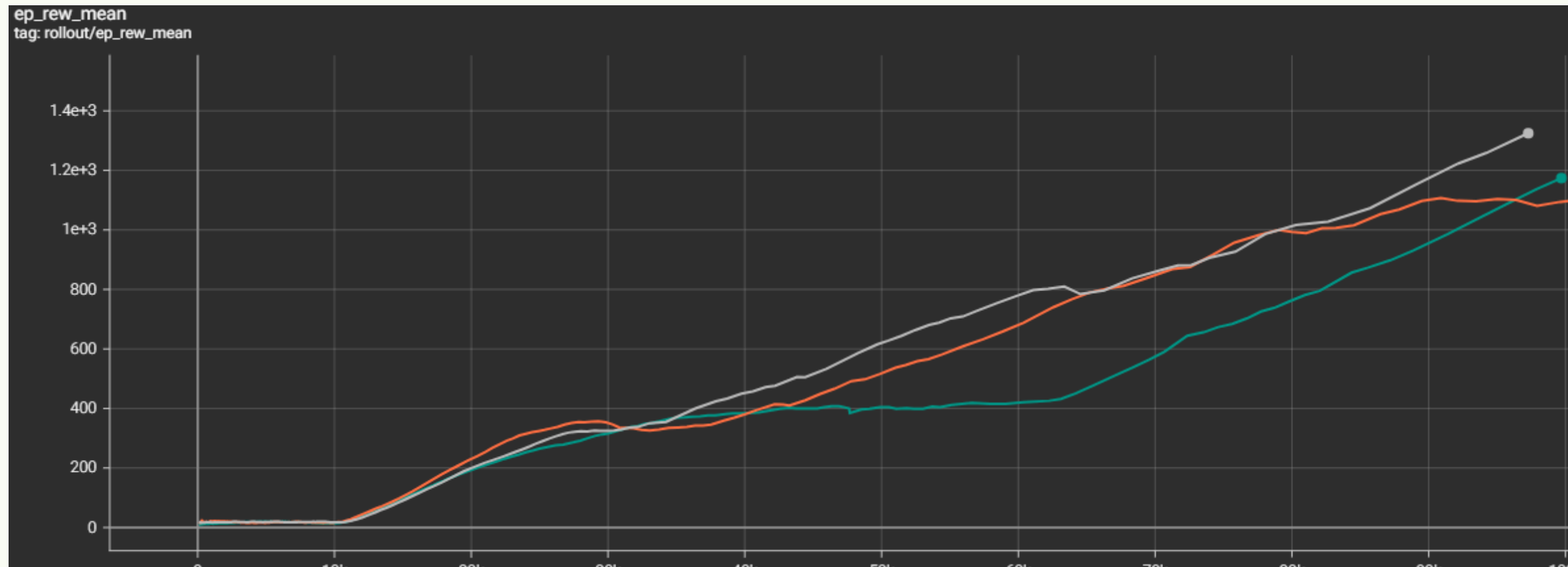
Hyperparameter Tuning: Learning Rate



Hyperparameter Tuning: Discount Factor



Hyperparameter Tuning: Replay Buffer Size



Optimal Hyper Parameters

Lr: 0.001

Gamma: 0.99

Buffer: 1,000,000

Avg Episodic Reward: 0.99

Reward Shaping

Rewards

total reward is: **reward** = *healthy_reward* + *forward_reward* - *ctrl_cost*.

healthy_reward: Every timestep that the Hopper is healthy (see definition in section "Episode End"), it gets a reward of fixed value `healthy_reward` (default is 1).

forward_reward: A reward for moving forward, this reward would be positive if the Hopper moves forward (in the positive x direction / in the right direction). $w_{forward} \times \frac{dx}{dt}$, where dx is the displacement of the "torso" ($x_{after-action} - x_{before-action}$), dt is the time between actions, which depends on the `frame_skip` parameter (default is 4), and `frametime` which is 0.002 - so the default is $dt = 4 \times 0.002 = 0.008$, $w_{forward}$ is the `forward_reward_weight` (default is 1).

ctrl_cost: A negative reward to penalize the Hopper for taking actions that are too large.

$w_{control} \times \|action\|_2^2$, where $w_{control}$ is `ctrl_cost_weight` (default is 10^{-3}).

```
torso_angle = obs[1]
joint_torques = action
velocity_penalty = -0.1 * abs(forward_velocity - 2.0)
stability_penalty = -0.1 * abs(torso_angle)
shaped_reward = reward + stability_penalty + velocity_penalty
```

Default Reward

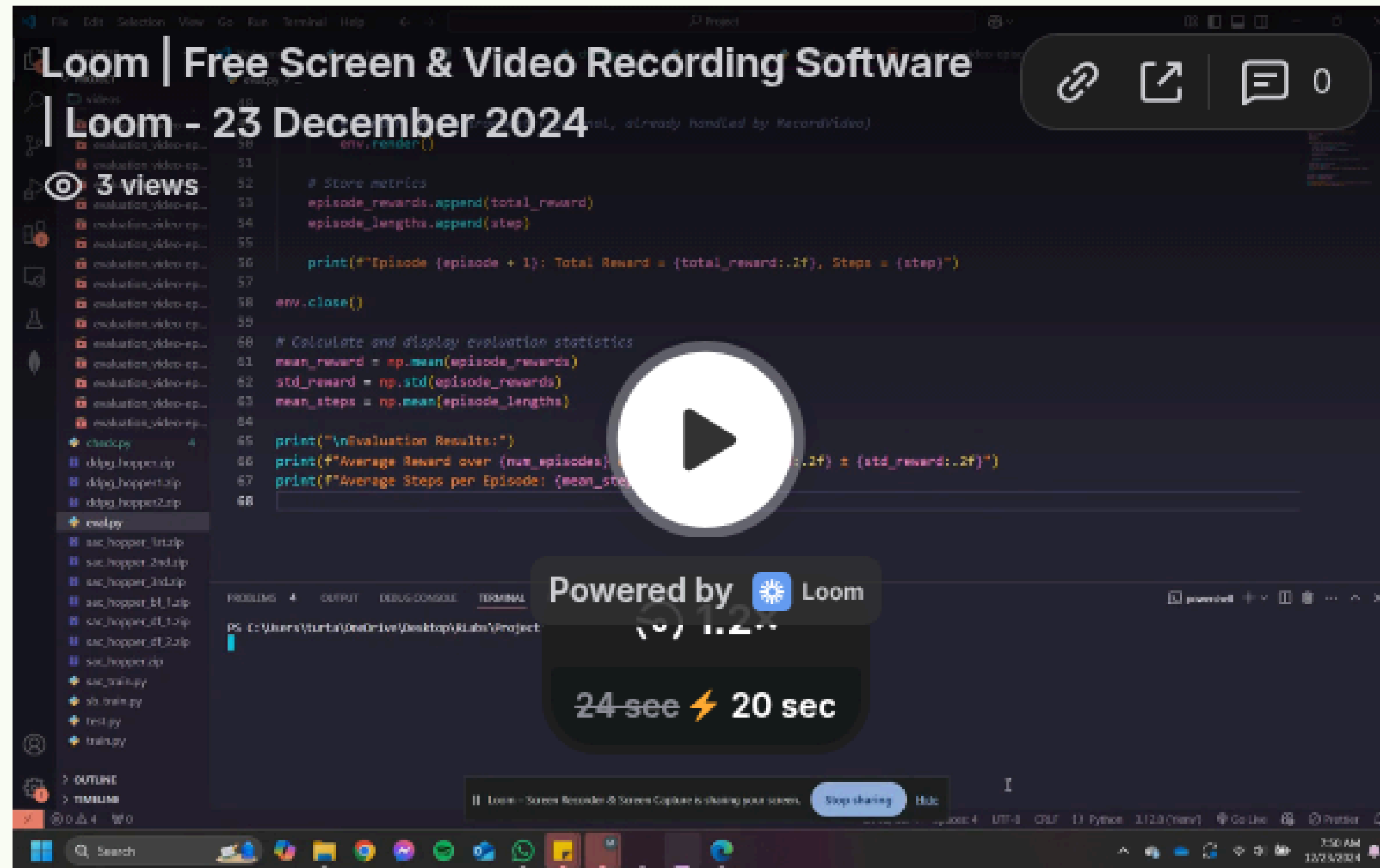
Reward = Healthy_Reward + Forward_Reward - Control_Cost

Shaped Reward

Add the following penalties to reward.

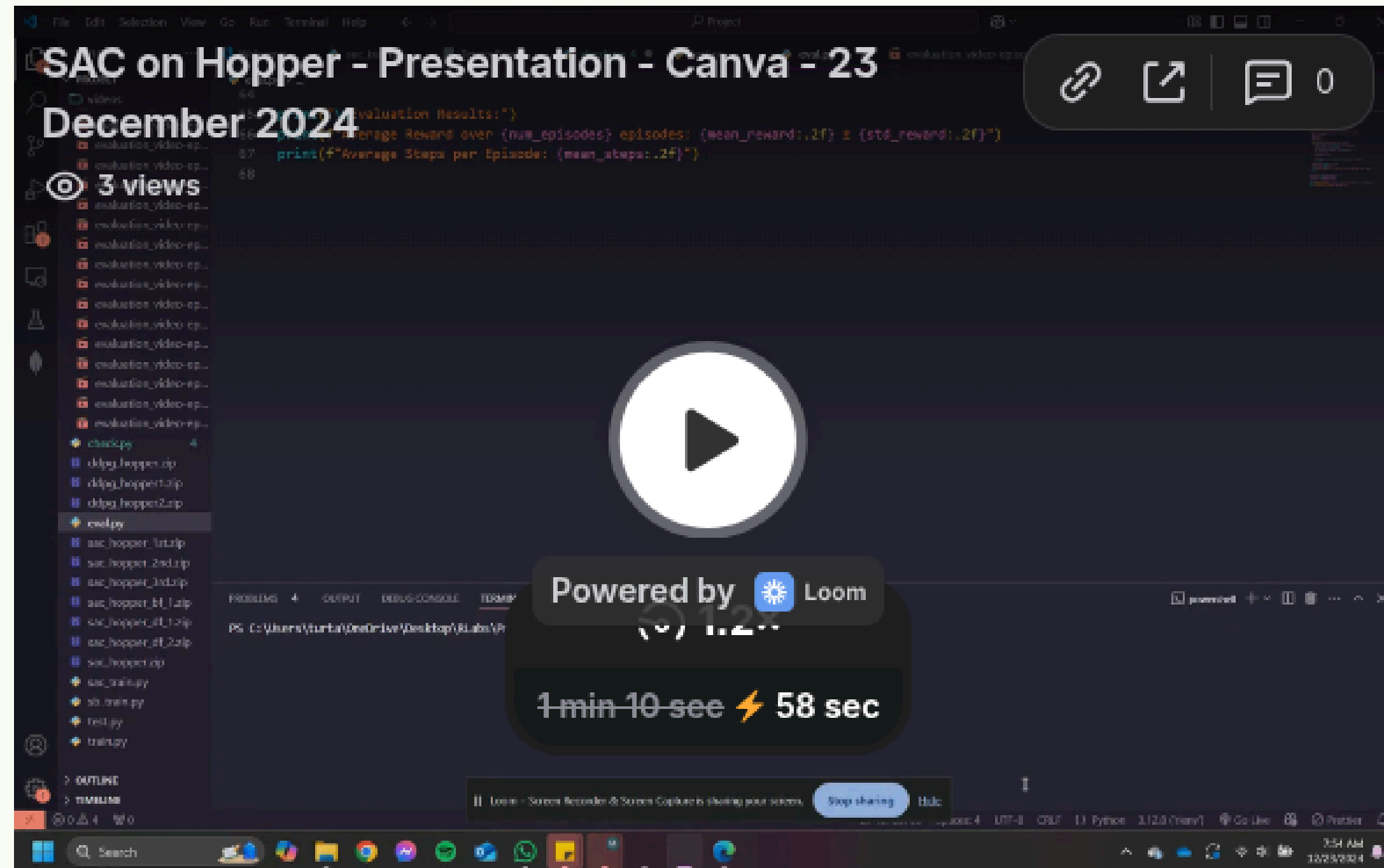
1. Excessive tilt of torso to encourage upright posture
2. Maintain a specific forward target velocity

Demo: Random Policy



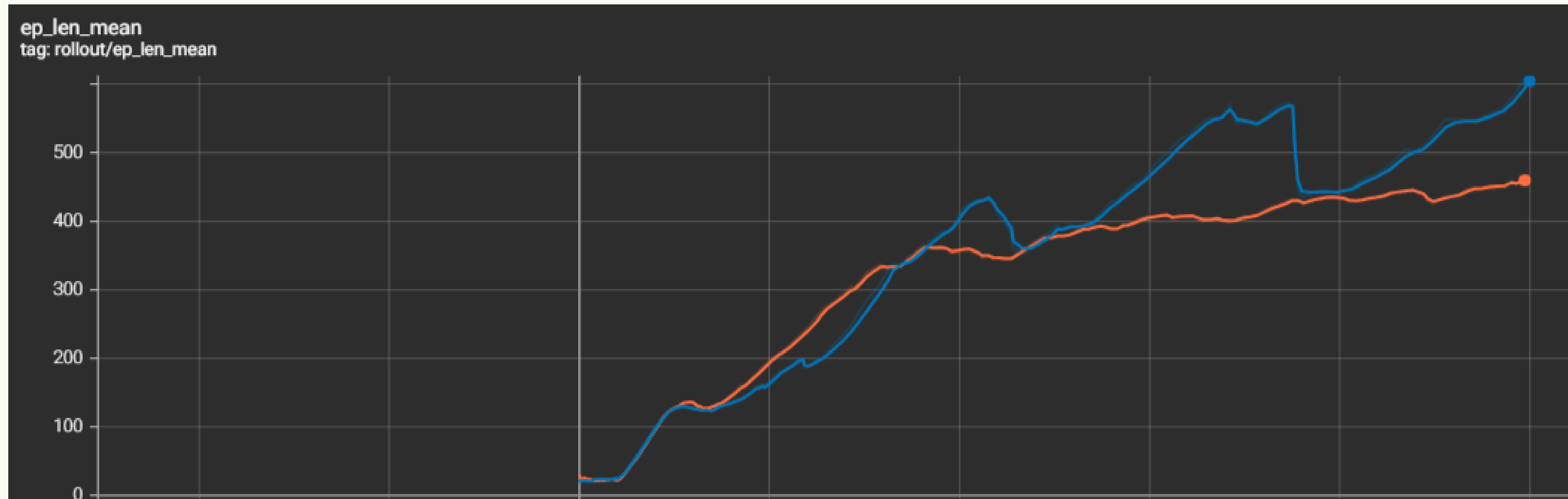
Average Reward over 10 episodes: 14.17 ± 5.92
Average Steps per Episode: 19.70

Demo: Optimal



Average Reward over 10 episodes: 1567.03 ± 54.23
Average Steps per Episode: 471.00

Demo: Shaped Reward



Demo: Shaped Reward

